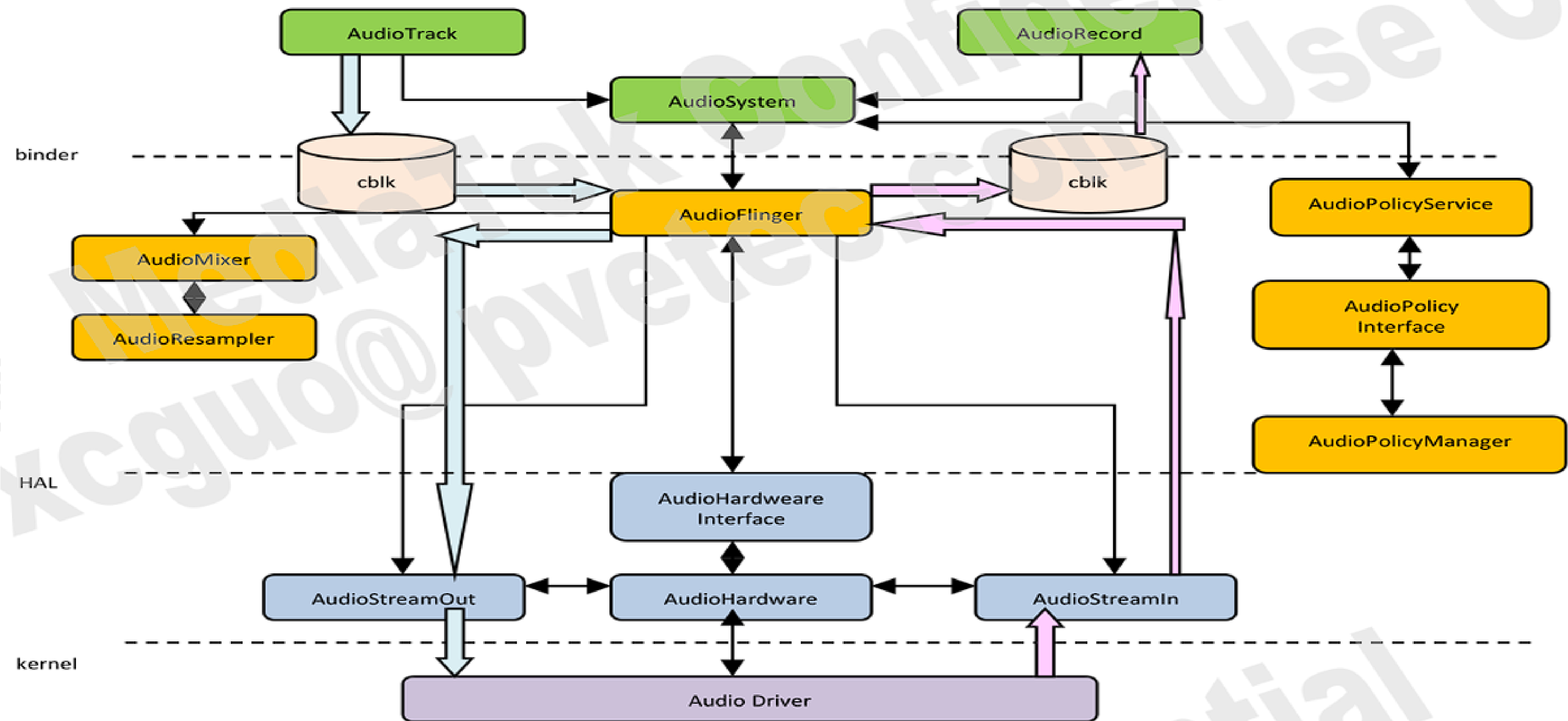


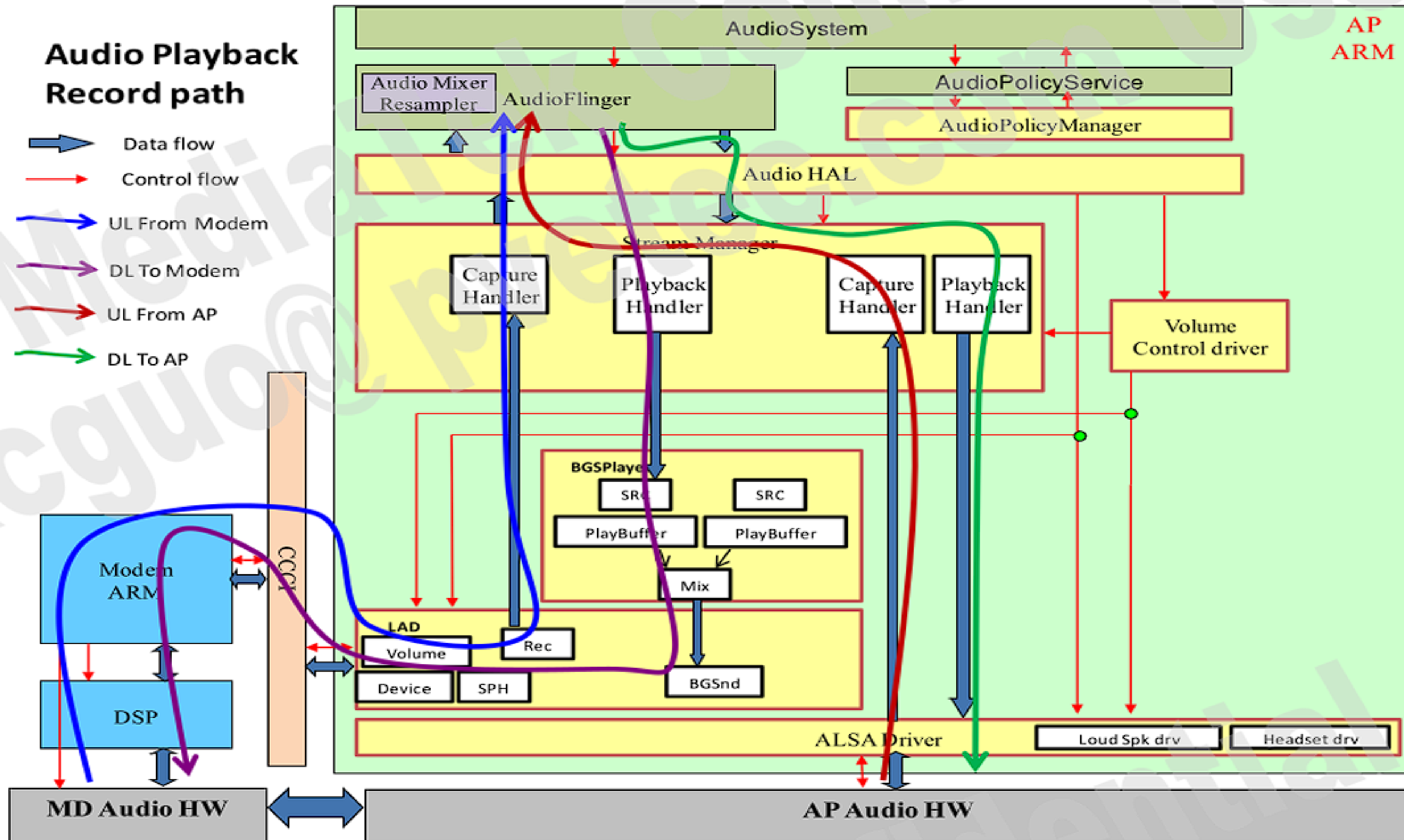
PVT 掌锐电子



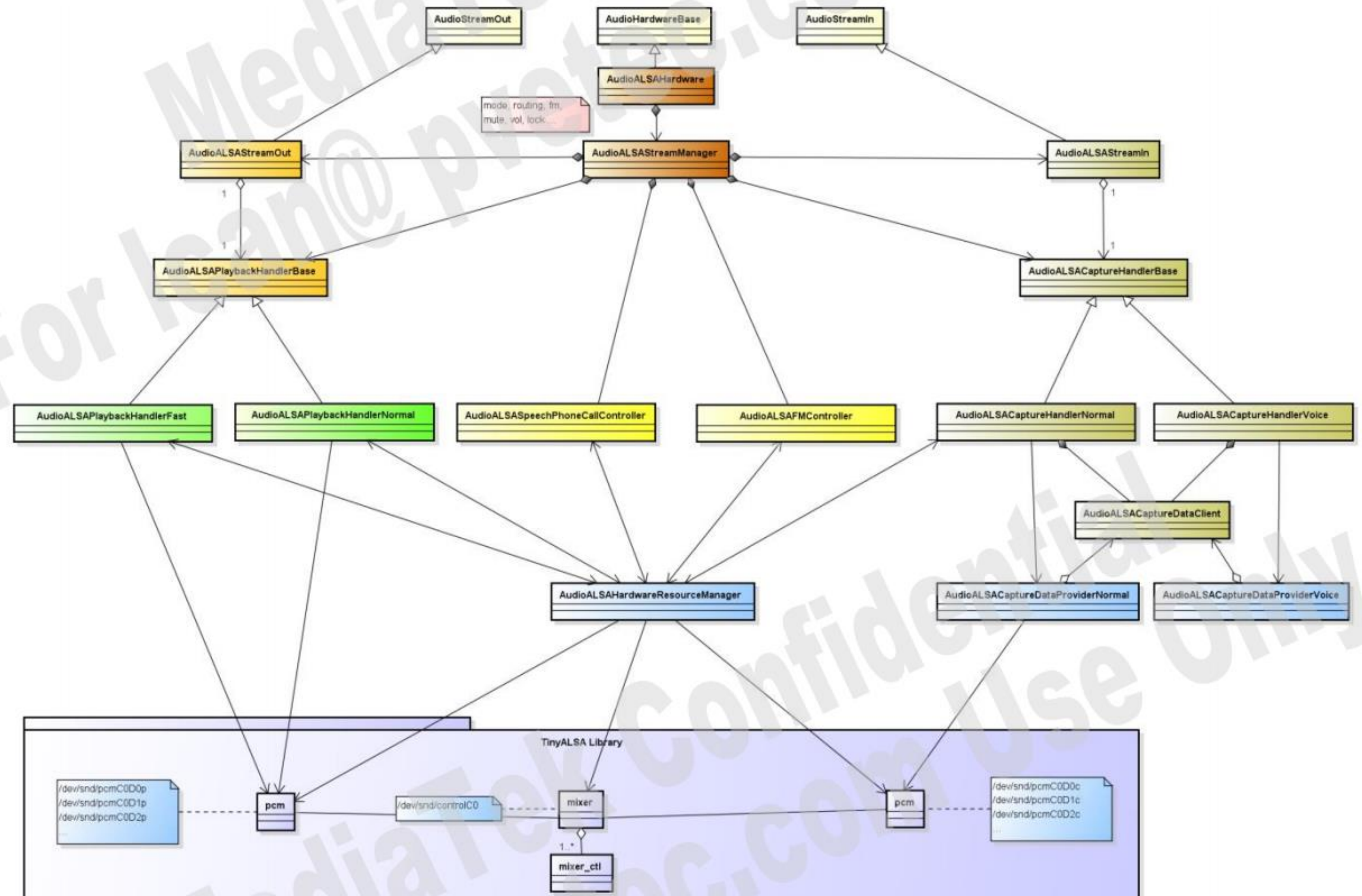
Audio软件架构



Audio数据流Flow



Audio Hal



HAL 的顶层控制接口是 `AudioALSAHardware`、`AudioALSAStrStreamOut` 和 `AudioALSAStrStreamIn`。这些类都使用 `AudioALSAStrStreamManager` 来控制音频模式、打开/关闭输入/输出流。因此 `AudioALSAStrStreamManager` 将拥有所有音频环境信息，如模式、路由、音量和静音信息。

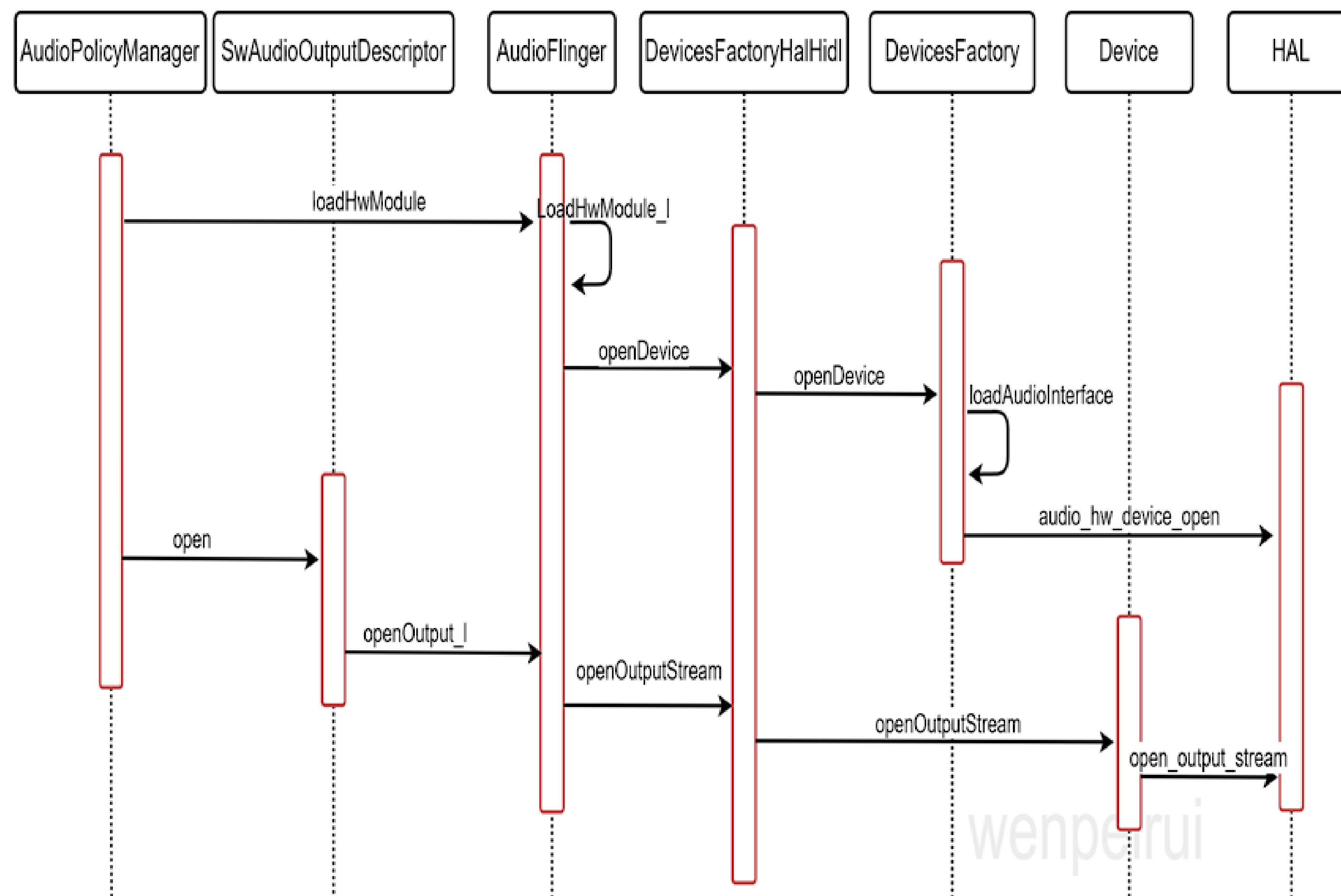
Audio Hal加载

HAL加载主要涉及以下几个模块的通信以及调用，如下图所示：

Framework:
AudioPolicyService、
AudioFlinger

HIDL: HalHidl

HAL: audio primary
hal

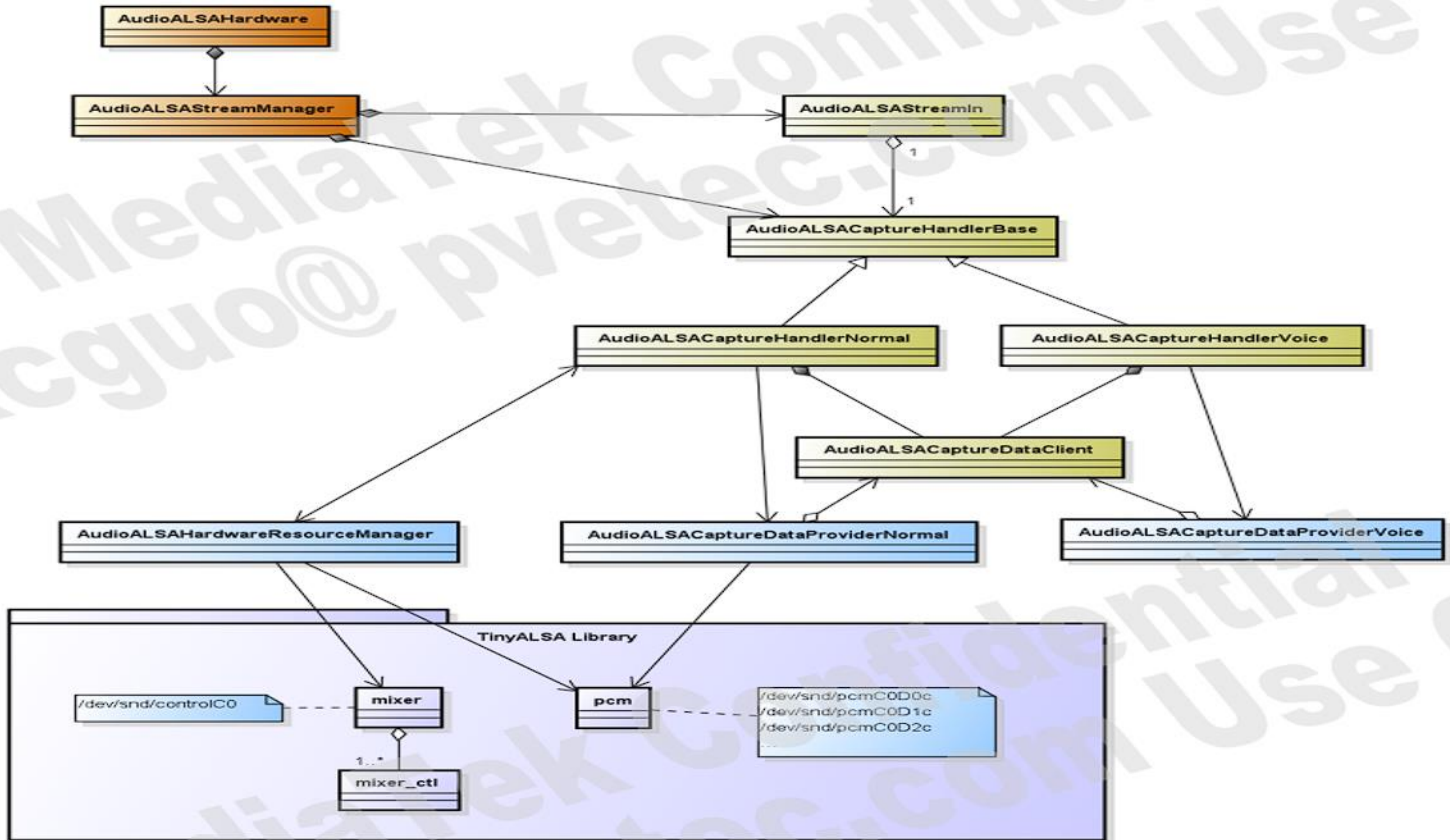


Audio 播放

AudioALSAStreamOut: 执行写入/待机/路由操作的类

- 不实现细节本身，而是使用 Playback Handler 执行实际打开/关闭/路由/写入操作
- 不同场景使用不同类型的 Playback Handler
- 第一次写入时，Stream Out 将调用 StreamManager 中的 createPlaybackHandler() 来获取 Playback Handler 的指针，在 standby() 时，destroyPlaybackHandler()
- 不要在 playback handler 中设置硬件寄存器，而是使用 TinyALSA 库来调用
- pcm_open()/pcm_close() 控制 AFE 硬件路径和内存设置
- mixer_open()/mixer_close() 获取各种类型的混音器控制以控制编解码器的驱动程序
- pcm_write() 将 PCM 数据写入 SRAM/DRAM

Audio 播放

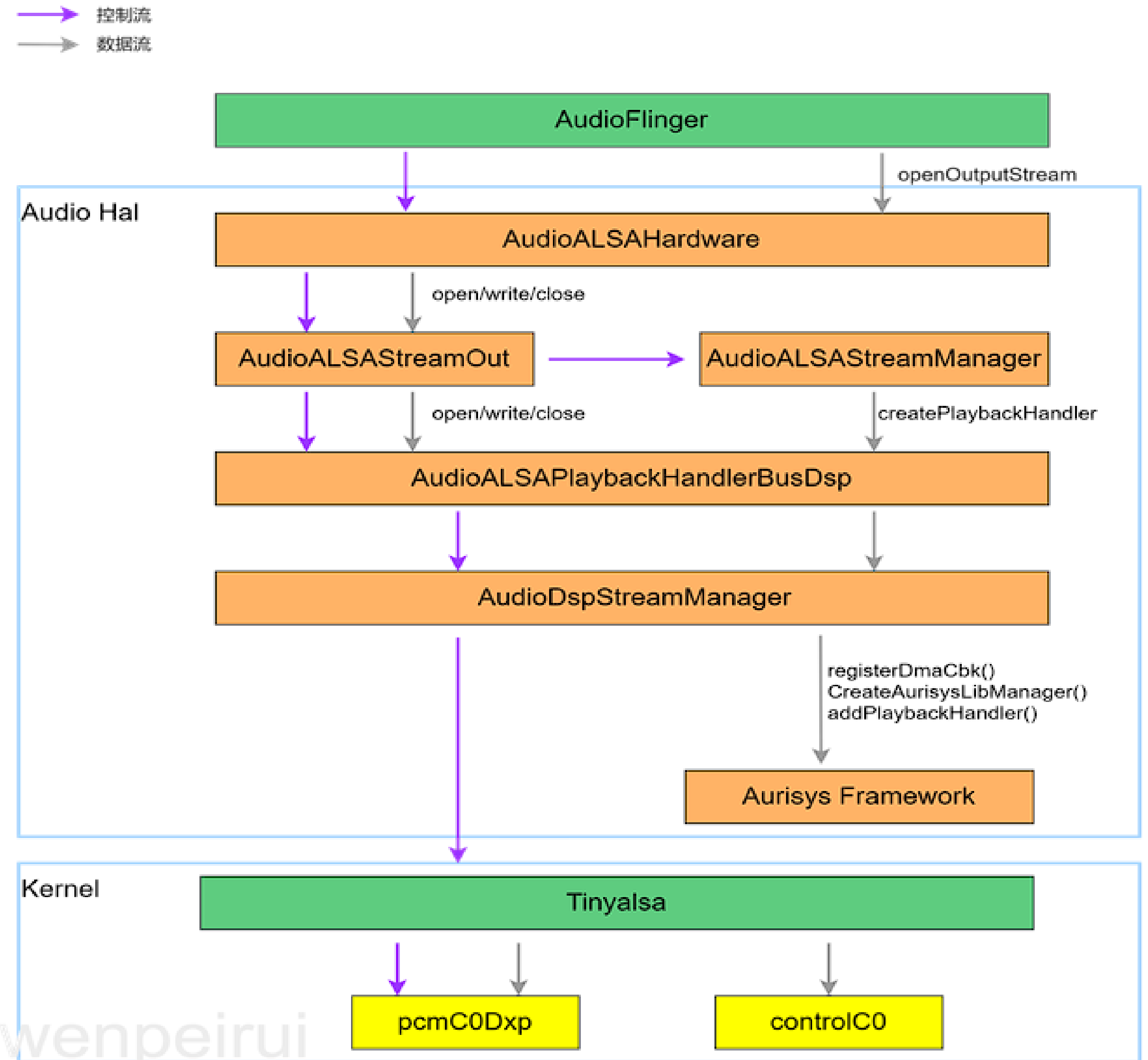


Audio 播放

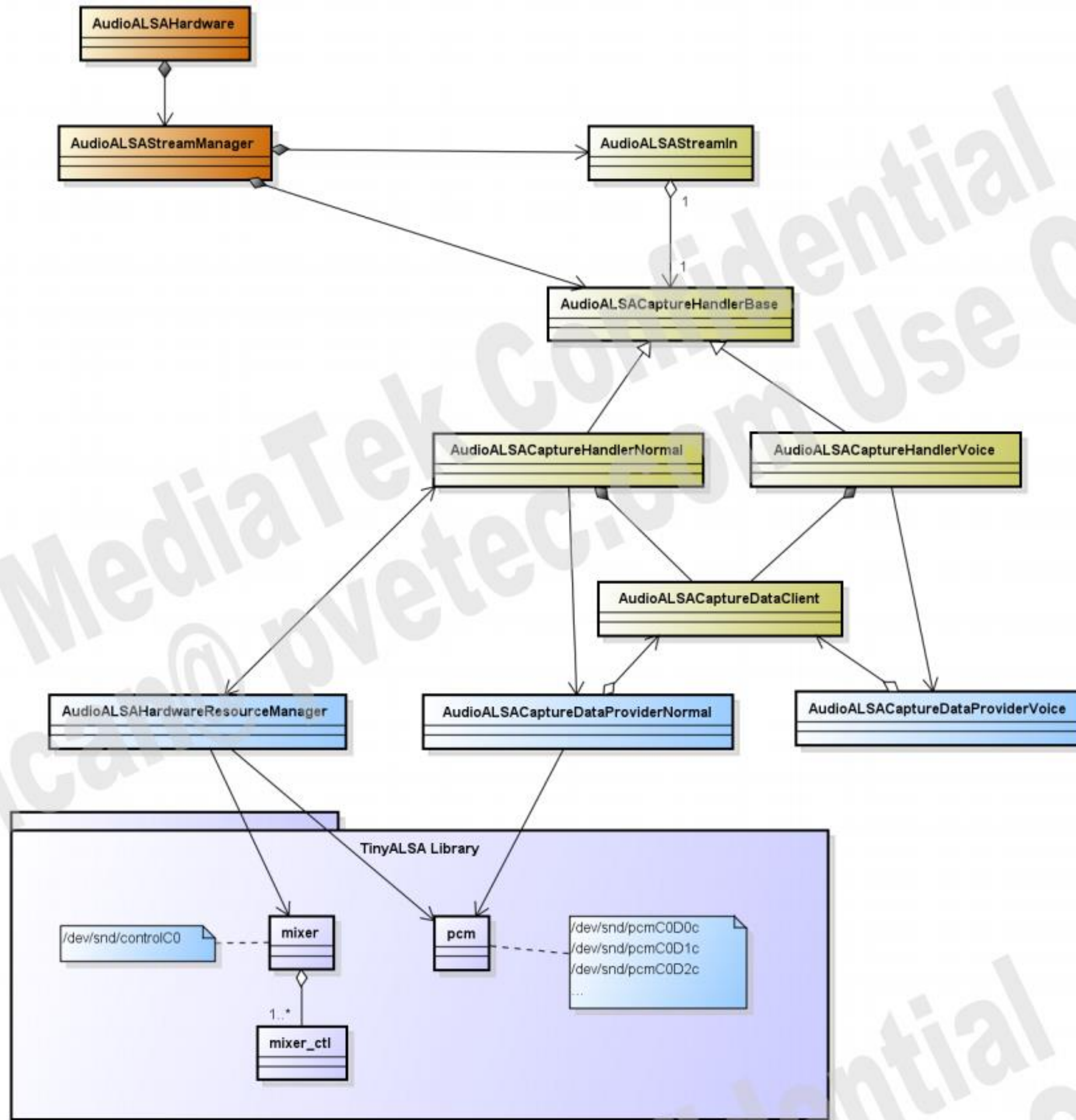
对于车载平台我们定义了MTK_AUDIO_AUTO_SUPPORT宏，使能了AudioALSAPlaybackHandlerBusDsp类，传入了stream_attribute_source参数对接上层的不同bus num。

播放开始时，AudioALSASStreamOut->open()函数后new AudioALSAPlaybackHandlerBusDsp成功后调用AudioALSAPlaybackHandlerBusDsp->open()函数，对音频流进行处理，并开始与ADSP通信。

播放结束后，framework会调用AudioALSASStreamOut->close()函数，释放AudioALSAPlaybackHandlerBusDsp类，以上是adsp handler的生命周期。



Audio 录音

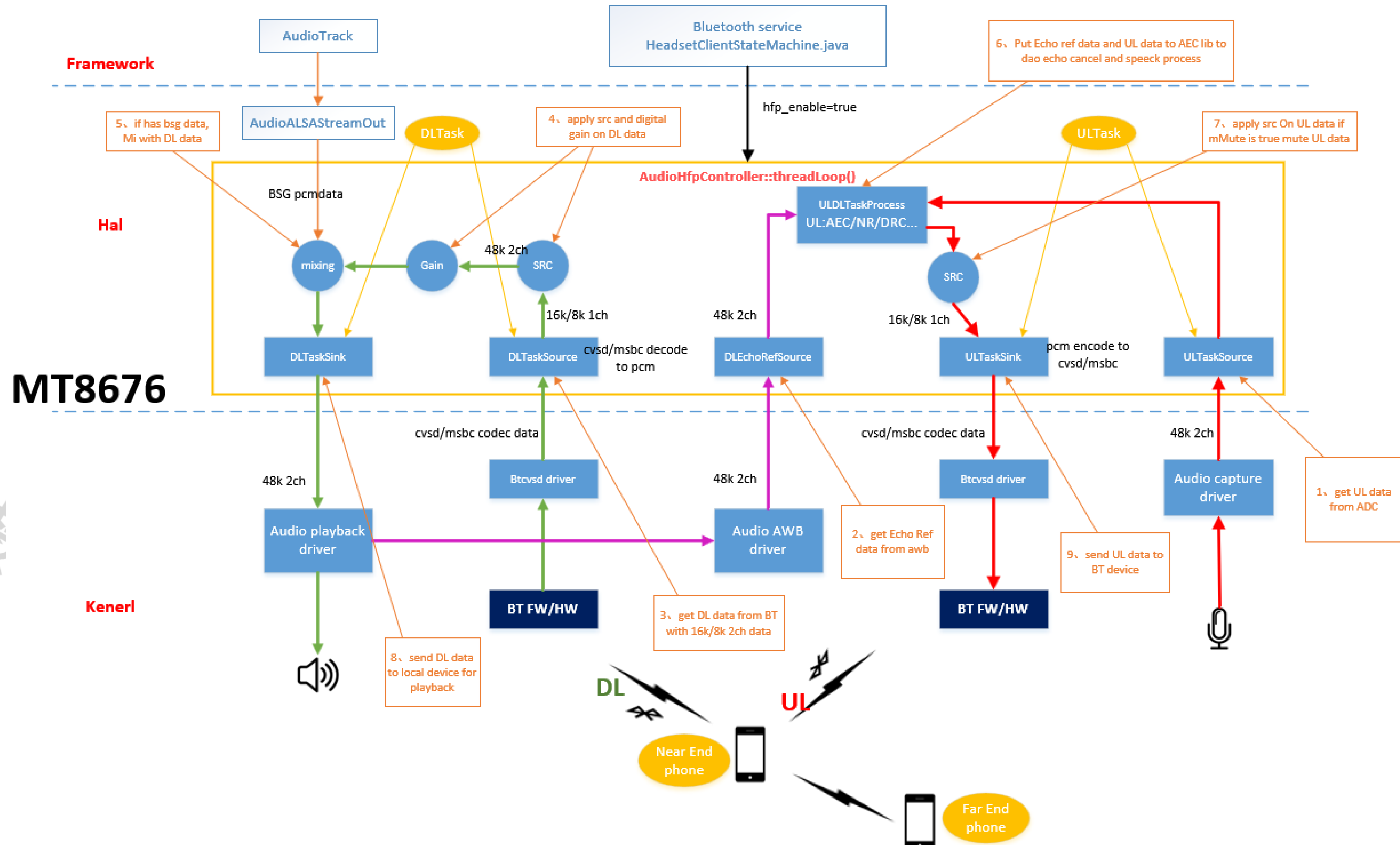


Audio 录音

AudioALSAStreamIn: 执行读取/待机/路由操作的类

- 不实现细节本身，而是使用 Capture Handler 绕过打开/关闭/路由/读取操作
- 不同场景使用不同类型的 Capture Handler
- Stream In 将在第一次 read()时调用 StreamManager 中的 createCaptureHandler()来获取 Capture Handler 的指针
- 此外，在 standby()时，destroyCaptureHandler()
- 不要在 Capture Handler 中设置硬件寄存器，而是使用 TinyALSA 库来调用
- pcm_open() /pcm_close()
 - 用于控制 AFE 硬件路径、内存设置
- mixer_open()/mixer_close()
 - 用于获取各种类型的混音器控制，如 Audio_ADC_1_Switch、Audio_ADC_2_Switch、Audio_Preampl1_Switch、Audio_Preampl2_Switch，以控制编解码器驱动程序
- pcm_read()
 - 用于将 PCM 数据读取到 SRAM/DRAM 中

Audio HFP 电话

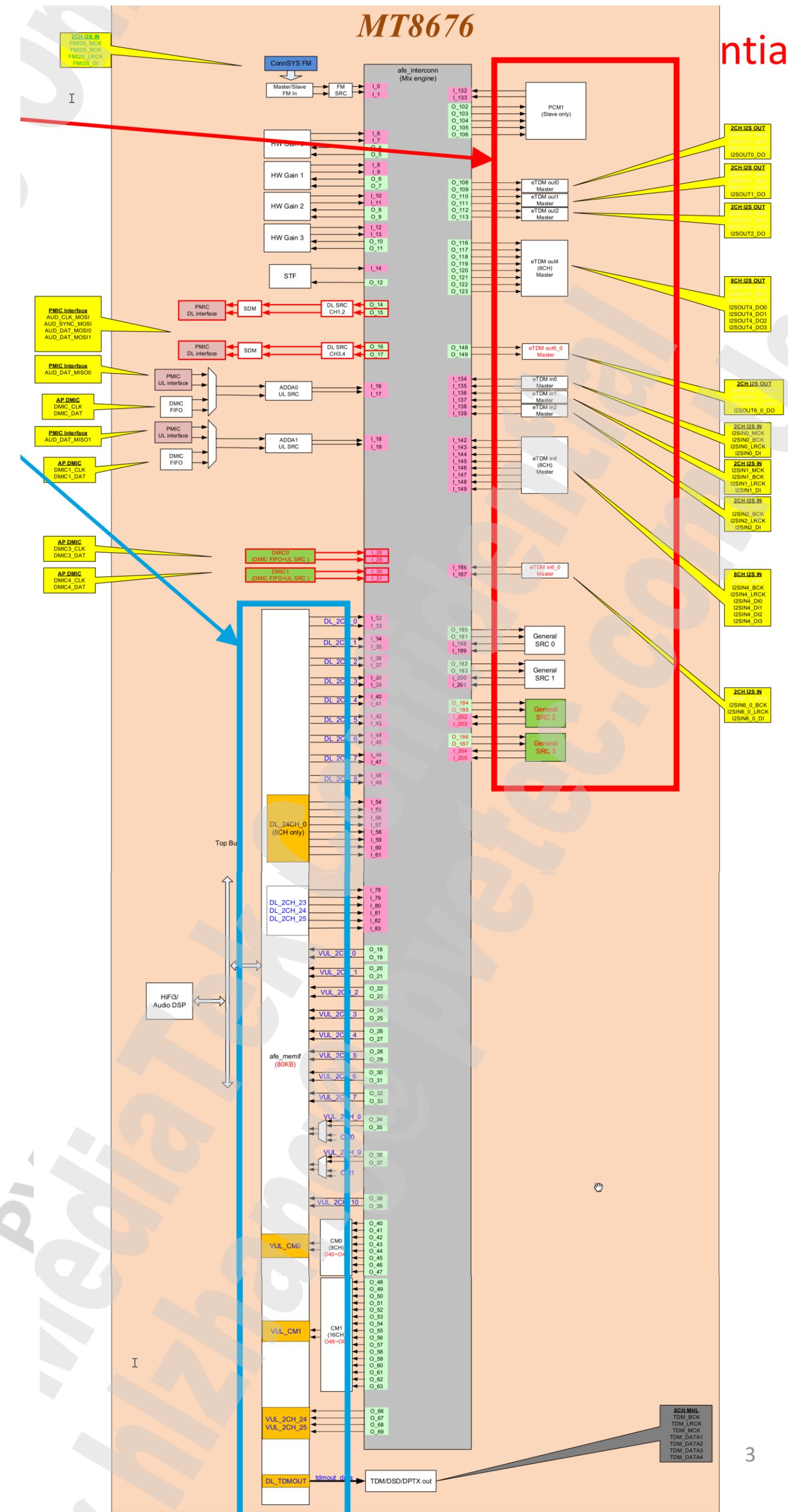


Audio AFE

MT8676 AFE HW Capability

- 下行**24ch**: 4组2ch I2S out, 1组8ch TDM out, 1组8ch Internal DP-TX out
- 上行**18ch**: 5组2ch I2S in, 1组8ch TDM in

In/Out	Max Valid Format	Mem IF (FE)	Audio IF (BE)
Max 24ch Output	2ch/192k/32b	DL_2ch_X	I2SOUT0: 2ch
	2ch/192k/32b	DL_2ch_X	I2SOUT1: 2ch
	2ch/192k/32b	DL_2ch_X	I2SOUT2: 2ch
	2ch/192k/32b	DL_2ch_X	I2SOUT6: 2ch
	8ch/48k/32b	DL_8CH_0 (8ch)	I2SOUT4: 8ch TDM
	8ch/96k/32b	DL_TDMOUT (8ch)	No PIN out, only for internal DP-TX
Max 18ch Input	2ch/192k/32b	VUL_2CH_X	FMI2S: 2ch master & slave
	2ch/192k/32b	VUL_2CH_X	I2SIN0: 2ch
	2ch/192k/32b	VUL_2CH_X	I2SIN1: 2ch
	2ch/192k/32b	VUL_2CH_X	I2SIN2: 2ch
	2ch/192k/32b	VUL_2CH_X	I2SIN6: 2ch
	8ch/48k/32b	VUL_CM0 (8ch)	I2SIN4: 8ch TDM



Audio AFE

2.1 MT8676 I2S 功能支持

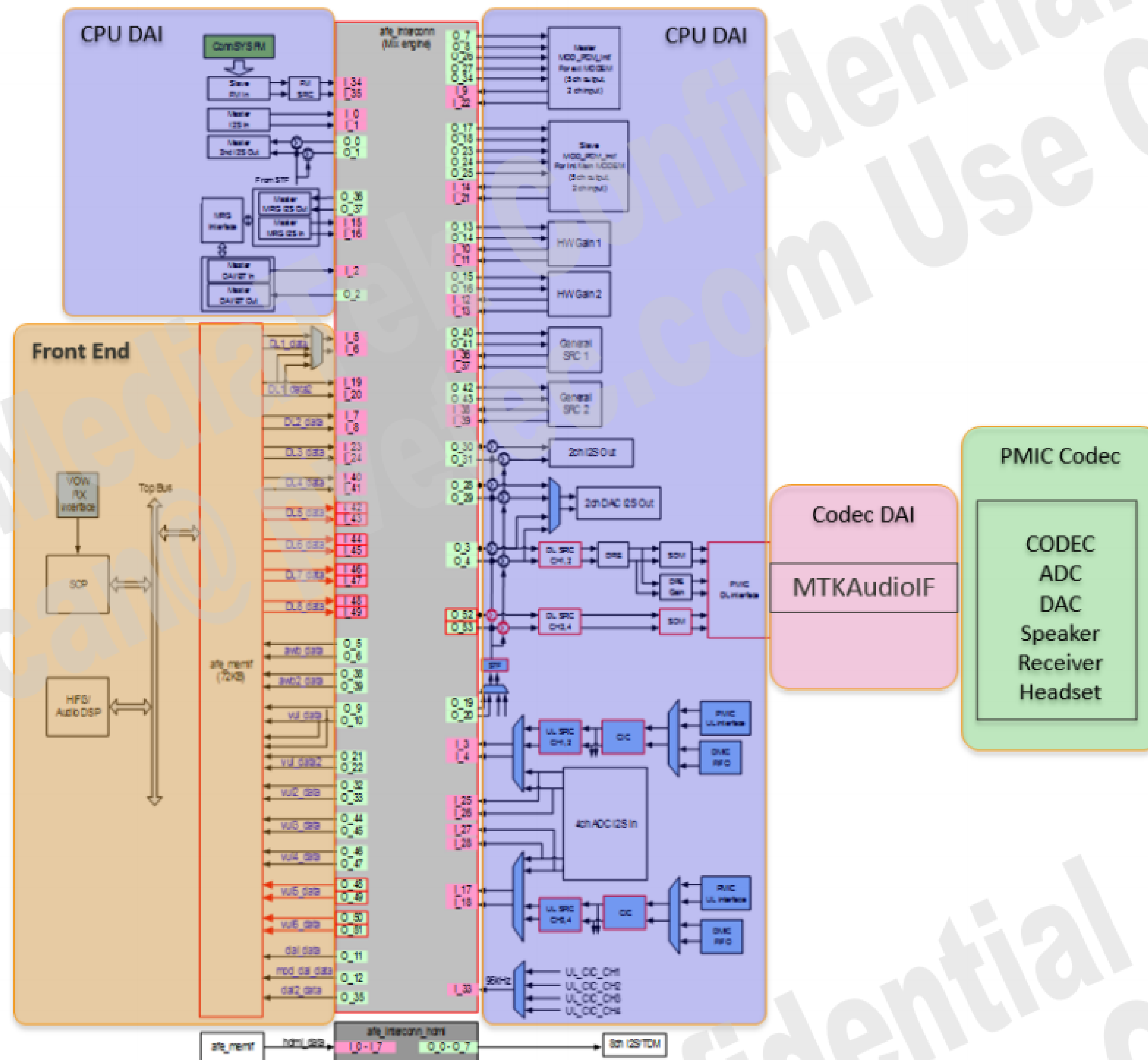
I2S 功能接口描述:

- MTK 音频接口
 - 适用于 MT6368 的 MTK 特定接口
- I2S0 – ADC/DAC 接口
 - 适用于任何音频接口
 - 支持**主输入**的标准 I2S
 - 输入能力高达 192kHz/32 位
 - 输出能力高达 384kHz/32 位
- I2S1 – ADC/DAC 接口
 - 支持**主输出**的标准 I2S
 - 最多 1 条数据线支持两个通道
 - 输入能力高达 192kHz/32 位
 - 输出能力高达 384k/32 位
- I2S2 – ADC/DAC 接口
 - 支持**主输入**的标准 I2S
 - 最多 2 条数据线支持四个通道
 - 输入能力高达 192kHz/32 位
 - 输出能力高达 384kHz/32 位
- I2S4 – ADC/DAC 接口
 - 支持**主输出**的标准 I2S
 - 最多 4 条数据线支持八个通道
 - 输入能力高达 192kHz/32 位
 - 输出能力高达 384kHz/32 位
- I2S6 – ADC/DAC 接口
 - 支持**主输入**的标准 I2S
 - 最多 1 条数据线支持两个通道
 - 输入能力高达 192kHz/32 位
 - 输出能力高达 384kHz/32 位

IP	Pin Name
I2S0	I2SIN0_LRCK / I2SIN0_BCK / I2S0_MCK / I2SIN0_DI/ I2SOUT0_DO
I2S1	I2SIN1_LRCK / I2SIN1_BCK / I2S1_MCK / I2SIN1_DI/ I2SOUT1_DO
I2S2	I2SIN2_LRCK / I2SIN2_BCK / I2SIN2_DI/ I2SOUT2_DO
I2S4	I2SIN4_LRCK / I2SIN4_BCK / I2SIN4_DIO/ I2SOUT4_DO0 / I2SIN4_DI1/ I2SOUT4_DO1 / I2SIN4_DI2/ I2SOUT4_DO2 / I2SIN4_DI3/ I2SOUT4_DO3
I2S6	I2SIN6_LRCK / I2SIN6_BCK / I2SIN6_DI/ I2SOUT6_DO
PMIC DL Interface	AUD_CLK_MOSI AUD_DAT_MOSIO AUD_DAT_MOSI1
PMIC UL Interface	AUD_CLK_MISO AUD_DAT_MISO0 AUD_DAT_MISO1

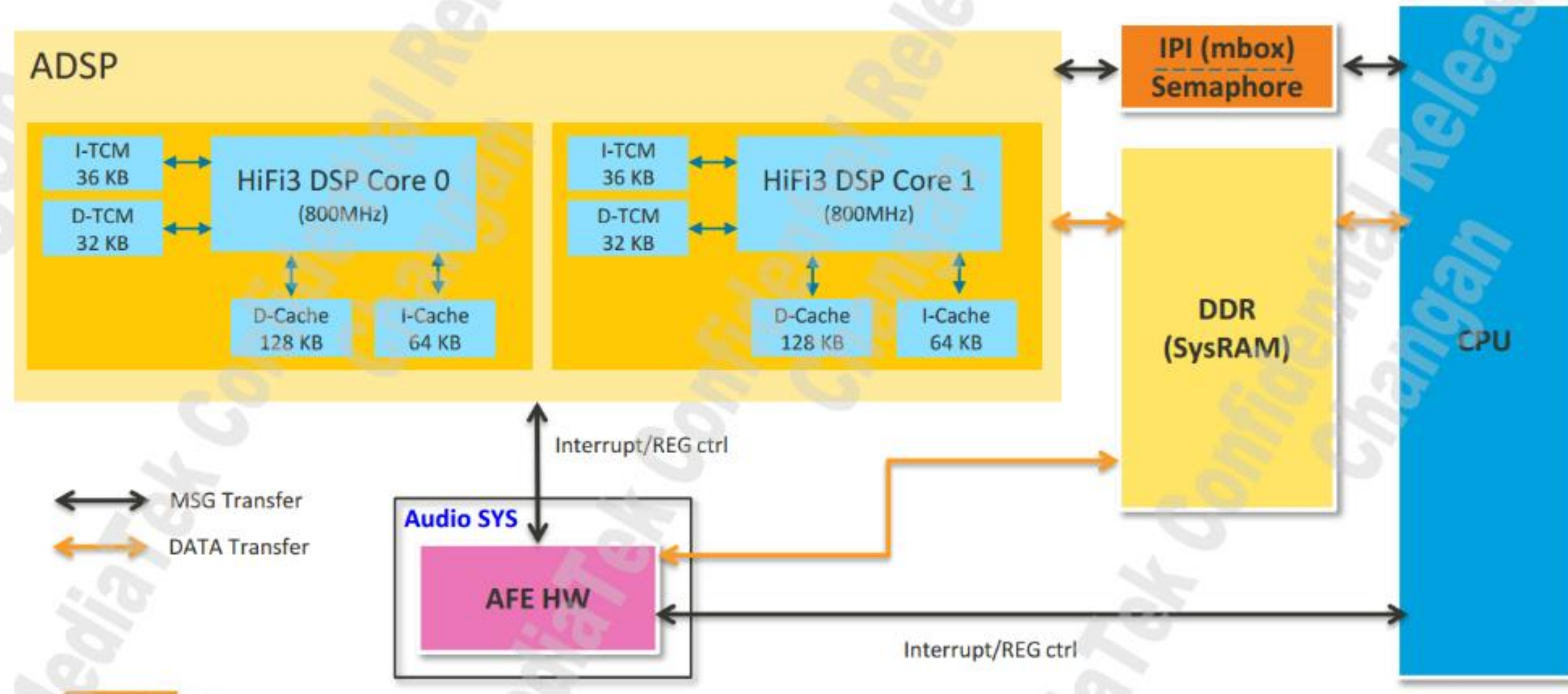
- MCLK 支持(I2S0/I2S1/I2S2/I2S4/I2S6)
 - 同步 MCLK 和 I2S BCK
 - 192k 模式下的能力高达 256fs
 - 96k 模式下的能力高达 512fs
 - 48k 模式下的能力高达 1024fs
 - 不支持 192fs 或 384fs 相关

以下是 MT8676 FE(memif)和 BE(ADDA/I2S/PCM/Hostless/..)框图:



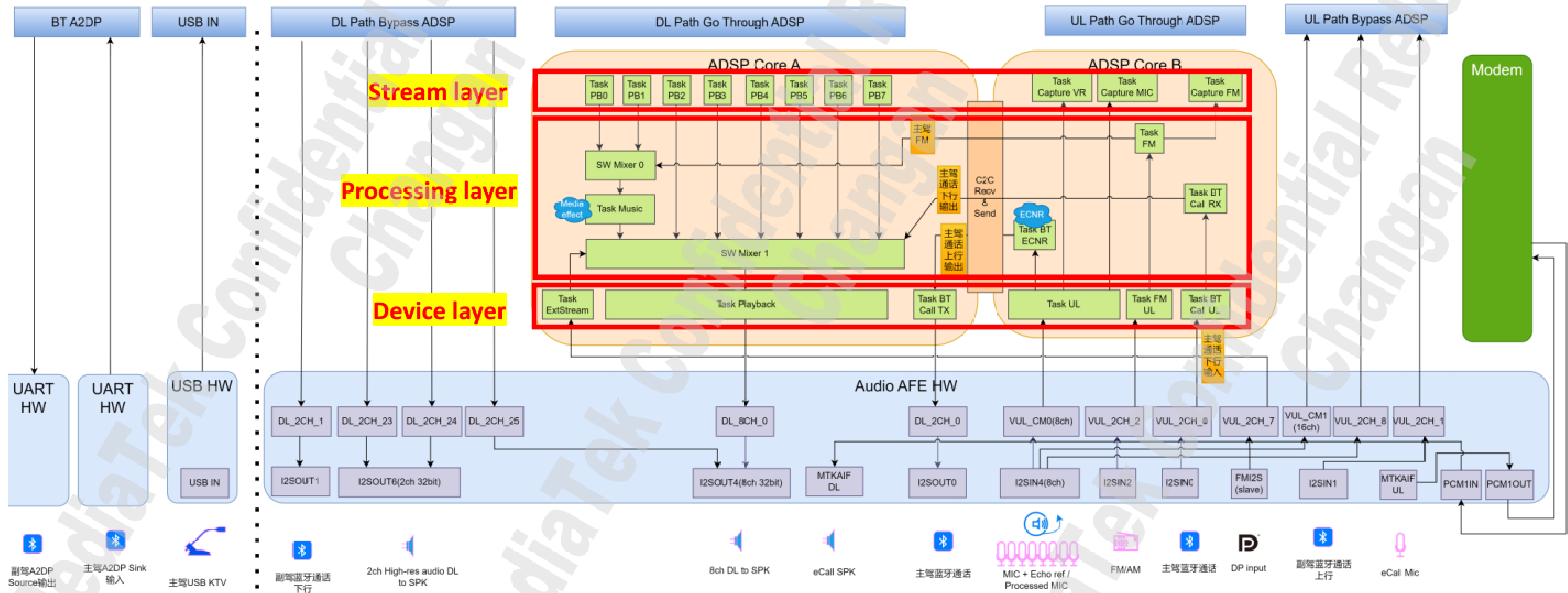
Audio ADSP

- HiFi3 Core 0 @800MHz
 - 64KB I-Cache
 - 128KB D-Cache
 - 36KB I-TCM
 - 32KB D-TCM
- HiFi3 Core 1 @800MHz
 - 64KB I-Cache
 - 128KB D-Cache
 - 36KB I-TCM
 - 32KB D-TCM
- Clock Rate: 800MHz *2

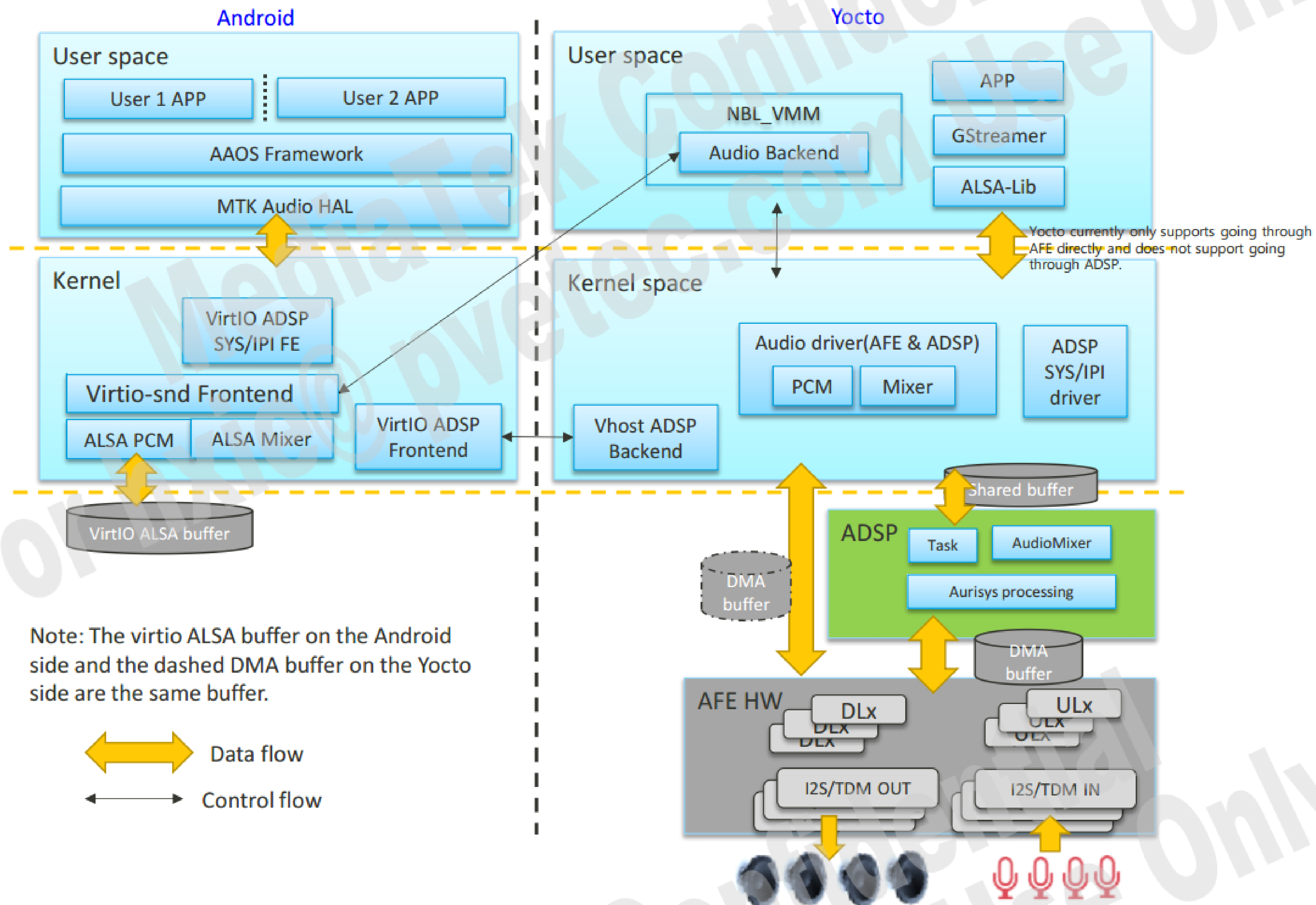


Audio ADSP

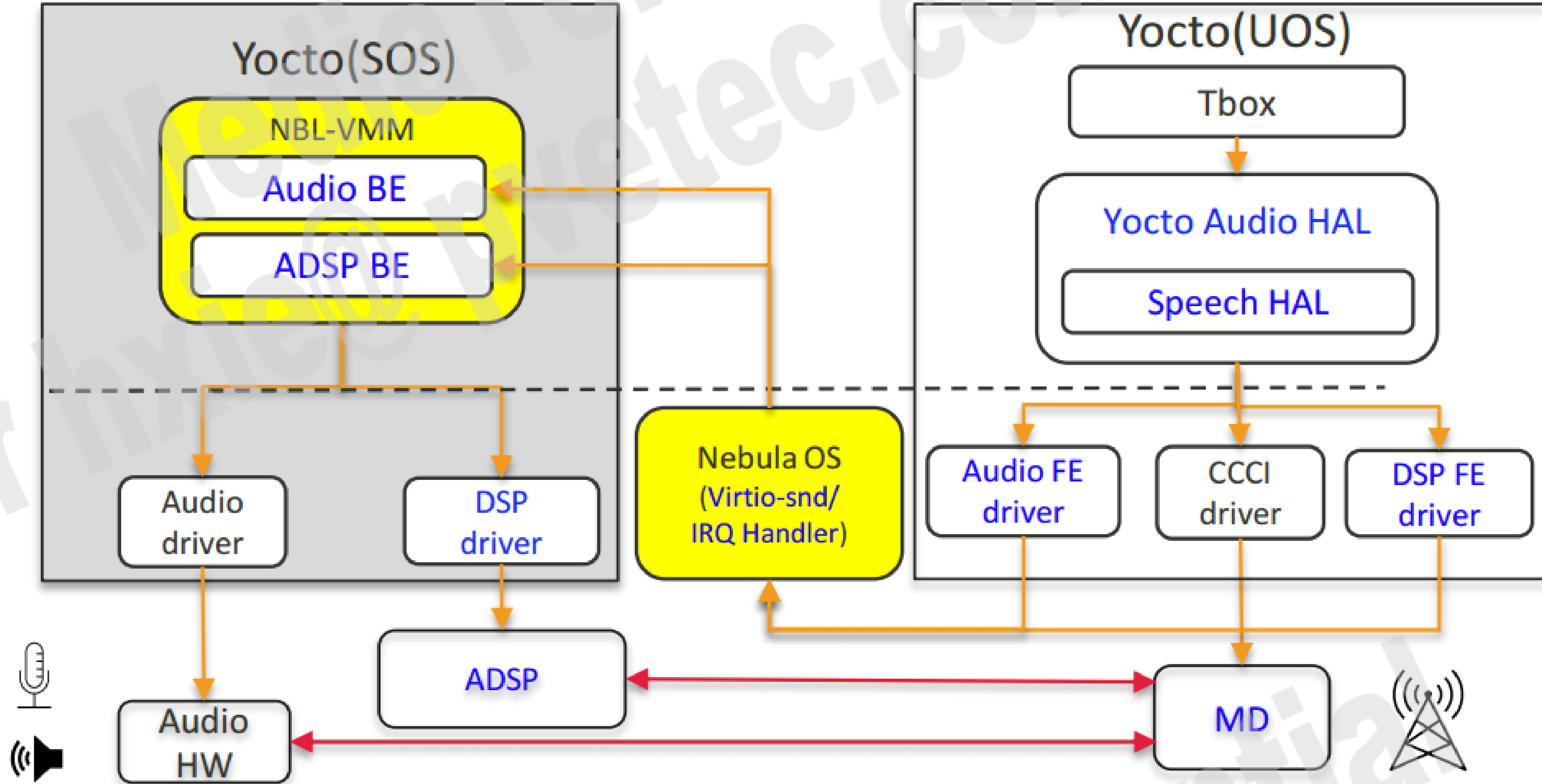
- Stream layer; Processing layer; Device layer



Audio 虚拟化



Audio 虚拟化



Audio 调试

1. Audio Log

在开发或者解决Bug的时候，需要打开Audio HAL或者Audio Driver的log，来定位问题点。

adb logcat 或是离线log包过滤AudioALSA和AudioDsp相关的log

2 使用debuglogger UI来录制Log:

MTK自带有debuglogger UI apk，可以用来录制log。测试前先启动log录制，测试完成后关闭log录制。然后在/data/debuglogger/目录下会看到有很多log生成。通过adb pull将其专区本地进行分析

3. 杂音、无声问题使用dump进行辅助分析

开DUMP

```
adb shell AudioSetParam SetDumpAudioStreamOut=268433409
```

```
adb shell AudioSetParam SetDumpAudioStreamIn=1
```

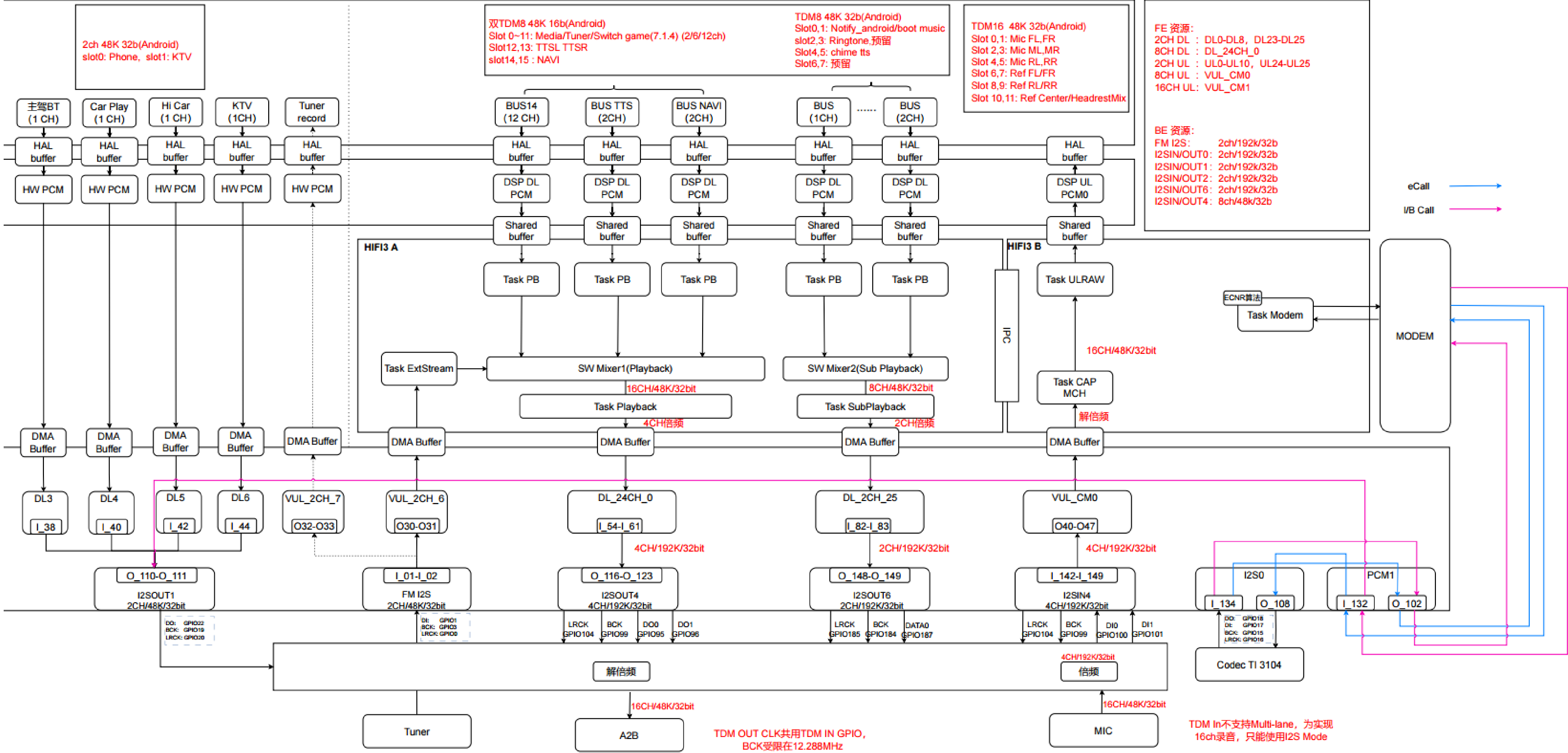
关DUMP

```
adb shell AudioSetParam SetDumpAudioStreamOut=0
```

```
adb shell AudioSetParam SetDumpAudioStreamIn=0
```

测试前，设置后这些属性，然后测试，测试结束会在/data/vendor/audiohal/audiodump/目录下生成dump文件，将这些文件pull到电脑上用音频分析软件进行分析

Audio 方案



Audio 方案

auto_audio_config.xml

```
<!-- busStreamConfig -->
<bus_stream_maps>
  <bus_stream bus_num="0" pcm_stream="DSP_Playback_Playback0"/>
  <bus_stream bus_num="1" pcm_stream="DSP_Playback_Playback1"/>
  <bus_stream bus_num="2" pcm_stream="DSP_Playback_Playback2"/>
  <bus_stream bus_num="3" pcm_stream="DSP_Playback_Playback3"/>
  <bus_stream bus_num="4" pcm_stream="Playback_9" low_latency="1">
    <device_path>
      <path name="PLAYBACK9_TO_ETDM_OUT"/>
    </device_path>
  </bus_stream>
  <bus_stream bus_num="5" pcm_stream="DSP_Playback_Playback5"/>
  <bus_stream bus_num="6" pcm_stream="DSP_Playback_Playback6"/>
  <bus_stream bus_num="7" pcm_stream="DSP_Playback_Playback7"/>
  <bus_stream bus_num="8" pcm_stream="DSP_Playback_Playback8"/>
  <bus_stream bus_num="9" pcm_stream="DSP_Playback_Playback9"/>
  <bus_stream bus_num="10" pcm_stream="DSP_Playback_Playback10"/>
  <bus_stream bus_num="11" pcm_stream="DSP_Playback_Playback11"/>
  <bus_stream bus_num="12" pcm_stream="DSP_Playback_Playback12"/>
  <bus_stream bus_num="13" pcm_stream="DSP_Playback_Playback13"/>
  <bus_stream bus_num="14" pcm_stream="DSP_Playback_Playback14"/>
  <bus_stream bus_num="15" pcm_stream="DSP_Playback_Playback15"/>
  <bus_stream bus_num="28" pcm_stream="DSP_Playback_Playback9"/>
  <bus_stream bus_num="29" pcm_stream="Playback_3" low_latency="1">
    <device_path>
      <path name="PLAYBACK3_TO_I2S1"/>
    </device_path>
  </bus_stream>

  <!-- DSP pcm stream without busnum to config dsp output path -->
  <bus_stream pcm_stream="DSP_Playback_Playback">
    <device_path>
      <path name="DL24_I2S4_RX"/>
    </device_path>
  </bus_stream>
  <bus_stream pcm_stream="DSP_Playback_Sub_Playback">
    <device_path>
      <path name="DL25_I2S6_RX"/>
    </device_path>
  </bus_stream>
</bus_stream_maps>
```

audio_device.xml

```
<path name="DL24_I2S4_RX" value="turnon">
  <kctl name="I2SOUT4_CH1 DL_24CH_CH1" value="1" />
  <kctl name="I2SOUT4_CH2 DL_24CH_CH2" value="1" />
  <kctl name="I2SOUT4_CH3 DL_24CH_CH3" value="1" />
  <kctl name="I2SOUT4_CH4 DL_24CH_CH4" value="1" />
</path>
<path name="DL24_I2S4_RX" value="turnoff">
  <kctl name="I2SOUT4_CH1 DL_24CH_CH1" value="0" />
  <kctl name="I2SOUT4_CH2 DL_24CH_CH2" value="0" />
  <kctl name="I2SOUT4_CH3 DL_24CH_CH3" value="0" />
  <kctl name="I2SOUT4_CH4 DL_24CH_CH4" value="0" />
</path>
<path name="DL25_I2S6_RX" value="turnon">
  <kctl name="I2SOUT6_CH1 DL25_CH1" value="1" />
  <kctl name="I2SOUT6_CH2 DL25_CH2" value="1" />
</path>
<path name="DL25_I2S6_RX" value="turnoff">
  <kctl name="I2SOUT6_CH1 DL25_CH1" value="0" />
  <kctl name="I2SOUT6_CH2 DL25_CH2" value="0" />
</path>
```

Audio 播放案例

```
private AudioTrack createStreamAudioTrack(int sampleRate, int channelConfig, int encoding) {
    // Start with buffer twice as large as needed.
    int bufferSizeBytes = 2 * AudioTrack.getMinBufferSize(sampleRate, channelConfig, encoding);
    AudioAttributes attributes = new AudioAttributes.Builder()
        .setUsage(AudioAttributes.USAGE_MEDIA)
        .setContentType(AudioAttributes.CONTENT_TYPE_MUSIC)
        .build();
    AudioFormat format = new AudioFormat.Builder()
        .setSampleRate(sampleRate)
        .setChannelMask(channelConfig)
        .setEncoding(encoding)
        .build();
    AudioTrack track = new AudioTrack.Builder()
        .setAudioAttributes(attributes)
        .setAudioFormat(format)
        .setTransferMode(AudioTrack.MODE_STREAM)
        .setBufferSizeInBytes(bufferSizeBytes)
        .build();
    return track;
}
```

CarAudioContext.java

```
private static final CarAudioContextInfo CAR_CONTEXT_INFO_MUSIC =
    new CarAudioContextInfo(new AudioAttributes[] {
        getAudioAttributeFromUsage(AudioAttributes.USAGE_UNKNOWN),
        getAudioAttributeFromUsage(AudioAttributes.USAGE_GAME),
        getAudioAttributeFromUsage(AudioAttributes.USAGE_MEDIA)
    }, "MUSIC", MUSIC);

private static final CarAudioContextInfo CAR_CONTEXT_INFO_NOTIFICATION =
    new CarAudioContextInfo(new AudioAttributes[] {
        getAudioAttributeFromUsage(AudioAttributes.USAGE_NOTIFICATION),
        getAudioAttributeFromUsage(AudioAttributes.USAGE_NOTIFICATION_EVENT)
    }, "NOTIFICATION", NOTIFICATION);
```

audio_policy_configuration.xml

```
<modules>
    <!-- Primary Audio HAL -->
    <module name="primary" halVersion="3.0">
        <attachedDevices>
            <item>bus0_media_out</item>
            <item>bus1_navigation_out</item>
        </attachedDevices>
        <defaultOutputDevice>bus0_media_out</defaultOutputDevice>
        <mixPorts>
            <mixPort name="mixport_bus0_media_out" role="source" flags="AUDIO_OUTPUT_FLAG_PRIMARY">
                <profile name="" format="AUDIO_FORMAT_PCM_32_BIT" samplingRates="48000" channelMasks="AUDIO_CHANNEL_OUT_STEREO"/>
                <profile name="" format="AUDIO_FORMAT_PCM_16_BIT" samplingRates="48000" channelMasks="AUDIO_CHANNEL_OUT_STEREO"/>
            </mixPort>
        </mixPorts>
        <devicePorts>
            <devicePort tagName="bus0_media_out" role="sink" type="AUDIO_DEVICE_OUT_BUS" address="bus0_media_out">
                <profile name="" format="AUDIO_FORMAT_PCM_32_BIT" samplingRates="48000" channelMasks="AUDIO_CHANNEL_OUT_STEREO"/>
                <gains>
                    <gain name="" mode="AUDIO_GAIN_MODE_JOINT" minValueMB="0" maxValueMB="300" defaultValueMB="300" stepValueMB="30"/>
                </gains>
            </devicePort>
        </devicePorts>
        <routes>
            <route type="mix" sink="bus0_media_out" sources="mixport_bus0_media_out"/>
        </routes>
    </module>
</modules>
```

car_audio_configuration.xml

```
<carAudioConfiguration version="3">
    <zones>
        <zone name="primary zone" isPrimary="true" audioZoneId="0" occupantZoneId="0">
            <zoneConfigs>
                <zoneConfig name="primary zone config speaker" isDefault="true">
                    <volumeGroups>
                        <group>
                            <device address="bus0_media_out">
                                <context context="music"/>
                            </device>
                        </group>
                        <group>
                            <device address="bus1_navigation_out">
                                <context context="navigation"/>
                            </device>
                        </group>
                    </volumeGroups>
                </zoneConfig>
            </zoneConfigs>
        </zone>
    </zones>
</carAudioConfiguration>
```

Audio 播放案例

audio_policy_configuration.xml

```
<mixPort name="mixport_bus14_multi_ch_out" role="source" flags="AUDIO_OUTPUT_FLAG_DIRECT">
  <profile name="" format="AUDIO_FORMAT_PCM_32_BIT"
    samplingRates="48000"
    channelMasks="AUDIO_CHANNEL_OUT_STEREO AUDIO_CHANNEL_OUT_7POINT1POINT4"/>
  <profile name="" format="AUDIO_FORMAT_PCM_16_BIT"
    samplingRates="48000"
    channelMasks="AUDIO_CHANNEL_OUT_STEREO AUDIO_CHANNEL_OUT_7POINT1POINT4"/>
</mixPort>
<devicePort tagName="bus14_multi_ch_out" role="sink" type="AUDIO_DEVICE_OUT_BUS"
  address="bus14_multi_ch_out">
  <profile name="" format="AUDIO_FORMAT_PCM_32_BIT"
    samplingRates="48000" channelMasks="AUDIO_CHANNEL_OUT_STEREO"/>
  <profile name="" format="AUDIO_FORMAT_PCM_32_BIT"
    samplingRates="48000" channelMasks="AUDIO_CHANNEL_OUT_7POINT1POINT4"/>
  <gains>
    <gain name="" mode="AUDIO_GAIN_MODE_JOINT"
      minValueMB="0" maxValueMB="300" defaultValueMB="300" stepValueMB="30"/>
  </gains>
</devicePort>
<route type="mix" sink="bus14_multi_ch_out" sources="mixport_bus14_multi_ch_out"/>
```

~~AudioALSASStreamManager.cpp~~

```
#ifndef MTK_ADSP_AUTO_SUPPORT
    if (AudioDspConfigManager::getInstance() -> isBusDspStream(stream_attribute_source->mBusNum)) {
        if (stream_attribute_source->mAudioOutputFlags == AUDIO_OUTPUT_FLAG_DIRECT)
            pPlaybackHandler = new AudioALSAPlaybackHandlerBusDirectDsp(stream_attribute_source); 多声道走Adsp
        else
            pPlaybackHandler = new AudioALSAPlaybackHandlerBusDsp(stream_attribute_source); 双声道走Adsp
        break;
    }
#endif
    pPlaybackHandler = new AudioALSAPlaybackHandlerBusNormal(stream_attribute_source); 不走Adsp
```

AudioDspConfigManager.cpp

```
const static std::map<std::string, uint8_t> pcmStreamToSceneMap = {
    {"DSP_Playback_Playback0", TASK_SCENE_PLAYBACK0},
    .....
    {"DSP_Playback_Playback", TASK_SCENE_AUDPLAYBACK},
    .....
};

const static std::map<std::string, uint8_t> nameToSceneMap = {
    {"sw_mixer_playback", TASK_SCENE_AUDPLAYBACK},
    ....
    {"sw_mixer_playback0", TASK_SCENE_PLAYBACK0},
};
```

audio_device.xml

```
auto audio config.xml
```

```
<root>
  <bus_stream_maps>
    <bus_stream bus_num="0" pcm_stream="DSP_Playback_Playback0"/>
    <bus_stream bus_num="7" pcm_stream="Playback_9" low_latency="1">
      <device_path>
        <path name="PLAYBACK9_TO_I256"/>
      </device_path>
    </bus_stream>
    <bus_stream bus_num="14" pcm_stream="DSP_Playback_Playback14"/>
    <bus_stream pcm_stream="DSP_Playback_Playback">
      <device_path>
        <path name="PLAYBACK11_TO_ETDM_OUT"/>
      </device_path>
    </bus_stream>
  </bus_stream_maps>

```

audio device.xml

```
<root>
  <!-- <card name="mt6897mt6338" />-->
  <card name="SoundCard" />

  <mixercontrol>
    <path name="PLAYBACK11_TO_ETDM_OUT" value="turnon">
      <kctl name="I2SOUT4_CH1 DL_24CH_CH1" value="1" />
      <kctl name="I2SOUT4_CH2 DL_24CH_CH2" value="1" />
      <kctl name="I2SOUT4_CH3 DL_24CH_CH3" value="1" />
      <kctl name="I2SOUT4_CH4 DL_24CH_CH4" value="1" />
      <kctl name="I2SOUT4_CH5 DL_24CH_CH5" value="1" />
      <kctl name="I2SOUT4_CH6 DL_24CH_CH6" value="1" />
      <kctl name="I2SOUT4_CH7 DL_24CH_CH7" value="1" />
      <kctl name="I2SOUT4_CH8 DL_24CH_CH8" value="1" />
    </path>
  </mixercontrol>
</root>
```

```
<sw_mixer_routes>  
  <sw_mixer_target name="sw_mixer_music" ch_in="16" ch_out="16" aurisys_on="0">  
    <sw_mixer_source name="sw_mixer_playback0">  
      <profile ch_in="2" ch_out="2" aurisys_on="0" ch_map="0x5555,0xaaaa"/>  
    </sw_mixer_source>  
  </sw_mixer_target>  
  
  <sw_mixer_target name="sw_mixer_playback" ch_in="16" ch_out="16" aurisys_on="1">  
    <sw_mixer_source name="sw_mixer_music">  
      <profile ch_in="16" ch_out="16" aurisys_on="1" ch_map="0x1,0x2,0x4,0x8,0x0x">  
      </sw_mixer_source>  
    </sw_mixer_target>  
  </sw_mixer_routes>  
  
root>
```

Audio 录音案例

```
public static AudioRecord createAudioRecord() {  
    return new AudioRecord.Builder()  
        .setAudioFormat(new AudioFormat.Builder()  
            .setSampleRate(SAMPLING_RATE_HZ)  
            .setEncoding(AudioFormat.ENCODING_PCM_16BIT)  
            .setChannelMask(AudioFormat.CHANNEL_IN_MONO).build())  
        .setAudioSource(MediaRecorder.AudioSource.DEFAULT)  
        .setBufferSizeInBytes(  
            AudioRecord.getMinBufferSize(SAMPLING_RATE_HZ,  
                AudioFormat.CHANNEL_IN_MONO,  
                AudioFormat.ENCODING_PCM_16BIT) * 10)  
        .build();  
}
```

enginedefault/src/Engine.cpp

```
switch (inputSource) {  
case AUDIO_SOURCE_DEFAULT:  
case AUDIO_SOURCE_MIC:  
    device = availableDevices.getDevice(  
        AUDIO_DEVICE_IN_BLUETOOTH_A2DP, String8(""), AUDIO_FORMAT_DEFAULT);  
    if (device != nullptr) break;  
    if (audio_is_bluetooth_out_sco_device(commDeviceType)) {  
        device = availableDevices.getDevice(  
            AUDIO_DEVICE_IN_BLUETOOTH_SCO_HEADSET, String8(""), AUDIO_FORMAT_DEFAULT);  
        if (device != nullptr) break;  
    }  
    if (FeatureOption::MTK_BLE_PHONECALL && commDeviceType == AUDIO_DEVICE_OUT_BUS) {  
        device = availableDevices.getDevice(  
            AUDIO_DEVICE_IN_BUS, String8(""), AUDIO_FORMAT_DEFAULT);  
        if (device != nullptr) break;  
    }  
    if (FeatureOption::MTK_BLE_PHONECALL || !audio_is_ble_out_device(commDeviceType)) {  
        device = availableDevices.getFirstExistingDevice({  
            AUDIO_DEVICE_IN_WIRED_HEADSET,  
            AUDIO_DEVICE_IN_USB_HEADSET, AUDIO_DEVICE_IN_USB_DEVICE,  
            AUDIO_DEVICE_IN_BUILTIN_MIC});  
    } else {  
        device = availableDevices.getFirstExistingDevice({  
            AUDIO_DEVICE_IN_WIRED_HEADSET,  
            AUDIO_DEVICE_IN_USB_HEADSET, AUDIO_DEVICE_IN_USB_DEVICE,  
            AUDIO_DEVICE_IN_BLE_HEADSET, AUDIO_DEVICE_IN_BLUETOOTH_BLE, AUDIO_DEVICE_IN_BUILTIN_MIC});  
    }  
}
```

audio_policy_configuration.xml

```
<module name="primary" halVersion="3.0">  
    <attachedDevices>  
        <item>Built-In Mic</item>  
    </attachedDevices>  
    <mixPorts>  
        <mixPort name="primary input" role="sink">  
            <profile name="" format="AUDIO_FORMAT_PCM_16_BIT" samplingRates="8000 11025 12000 16000 22050  
24000 32000 44100 48000" channelMasks="AUDIO_CHANNEL_IN_MONO AUDIO_CHANNEL_IN_STEREO AUDIO_CHANNEL_IN_FRONT_BACK"/>  
        </mixPort>  
    </mixPorts>  
    <devicePorts>  
        <devicePort tagName="Built-In Mic" type="AUDIO_DEVICE_IN_BUILTIN_MIC" role="source">  
            <profile name="" format="AUDIO_FORMAT_PCM_16_BIT" samplingRates="8000 11025 12000 16000 22050  
24000 32000 44100 48000" channelMasks="AUDIO_CHANNEL_IN_MONO AUDIO_CHANNEL_IN_STEREO AUDIO_CHANNEL_IN_FRONT_BACK"/>  
        </devicePort>  
    </devicePorts>  
    <routes>  
        <route type="mix" sink="primary input" sources="Built-In Mic,Built-In Back Mic"/>  
    </routes>  
</module>
```

Audio 录音案例

```
1 AudioALSASStreamManager.cpp
2
3 AudioALSASStreamManager::createCaptureHandler()
4
5 #ifdef MTK_AUDIO_AUTO_SUPPORT           可客制化判断条件
6     if (1) (((stream_attribute_target->input_source == AUDIO_SOURCE_CUSTOMIZATION1) ||
7             //((stream_attribute_target->input_source == AUDIO_SOURCE_VOICE_RECOGNITION) &&
8             // property_get_bool("vendor.audio.capturemch.enable", false)))
9         pCaptureHandler = new AudioALSACaptureHandlerNormal(stream_attribute_target);
10    else                                   可客制化使用不同的handler
11 #endif
```

AudioALSACaptureHandlerNormal.cpp

AudioALSACaptureHandlerNormal::open()

```
#ifdef MTK_AUDIO_AUTO_SUPPORT           可客制化判断条件
    if (mStreamAttributeTarget->input_source == AUDIO_SOURCE_VOICE_RECOGNITION)
    {
        mCaptureDataClient = new           可客制化使用不同的provider
        AudioALSACaptureDataClientChannelSelector(AudioALSACaptureDataProviderAfeMch::getInstance(),
                                                    可客制化使用不同的client
                                                    mStreamAttributeTarget);
    } else
#endif

#ifdef MTK_ADSP_AUTO_SUPPORT
    if(1) // ((mStreamAttributeTarget->input_source == AUDIO_SOURCE_VOICE_RECOGNITION) &&
        //property_get_bool("vendor.audio.capturemch.enable", false))
    {
        mCaptureDataClient = new
        AudioALSACaptureDataClientChannelSelector(AudioALSACaptureDataProviderDspMch::getInstance(),

        mStreamAttributeTarget);
    } else
#endif
```

Audio 录音案例

AudioALSCaptureDataProviderAfeMch.cpp

AudioALSCaptureDataProviderAfeMch::open()

```
int pcmindex = AudioALSADeviceParser::getInstance()->GetPcmIndexByString(keypcmCapture10);  
int cardindex = AudioALSADeviceParser::getInstance()->GetCardIndexByString(keypcmCapture10);  
  
mApTurnOnSequence = AUDIO_CTL_I2S2_TO_CAPTURE10_4CH;  
AudioALSADeviceConfigManager::getInstance()->ApplyDeviceTurnonSequenceByName(mApTurnOnSequence);  
  
/* workaround: disable dsp share mem befor pcm open */  
AudioDspStreamManager::getInstance()->setAfeInDspShareMem(TASK_SCENE_CAPTURE_UL1, false, false);  
AudioDspStreamManager::getInstance()->setDspRuntimeEn(TASK_SCENE_CAPTURE_UL1, false);  
  
int ret = pthread_create(&hReadThread, NULL, AudioALSCaptureDataProviderAfeMch::readThread, (void *)this);
```

AudioALSCaptureDataProviderAfeMch::readThread()

```
ret = pDataProvider->pcmRead(pDataProvider->mPcm, linear_buffer, kReadBufferSize);  
  
// use ringbuf format to save buffer info  
pDataProvider->mPcmReadBuf.pBufBase = linear_buffer;  
pDataProvider->mPcmReadBuf.bufLen = Read_Size + 1; // +1: avoid pRead == pWrite  
pDataProvider->mPcmReadBuf.pRead = linear_buffer;  
pDataProvider->mPcmReadBuf.pWrite = linear_buffer + Read_Size;  
pDataProvider->provideCaptureDataToAllClients(open_index);
```

AudioALSCaptureDataProviderBase.cpp

```
AudioALSCaptureDataProviderBase::provideCaptureDataToAllClients(const uint32_t open_index)  
{  
    for (size_t i = 0; i < mCaptureDataClientVector.size(); i++) {  
        pCaptureDataClient = mCaptureDataClientVector[i];  
        pCaptureDataClient->copyCaptureDataToClient(mPcmReadBuf);  
    }  
}
```

AudioALSCaptureDataClientChannelSelector.cpp

AudioALSCaptureDataClientChannelSelector::AudioALSCaptureDataClientChannelSelector()

```
mCaptureDataProvider->configStreamAttribute(mStreamAttributeTarget); // config attribute for input device  
mCaptureDataProvider->attach(this, _IsLowLatencyCapture(), mStreamAttributeTarget, NULL, &startFrameCount,  
NULL);
```

AudioALSCaptureDataClientChannelSelector::copyCaptureDataToClient() 从provider给回到client的函数

```
char *pRawDataLinearBuf = new char[kNumRawData];  
RingBuf_copyToLinear(pRawDataLinearBuf, &mRawDataBuf, kNumRawData); //原始数据ringbuffer  
// (mRawDataBuf) 给到pRawDataLinearBuf  
  
// 3. Resample using mBliSrc, pRawDataLinearBuf -> pSrcDataLinearBuf  
if (mUseMultiChannelSrc) {  
    mBliSrc->multiChannelProcess((int32_t *)pRawDataLinearBuf, &kNumRawData,  
                                (int32_t *)pSrcDataLinearBuf, &num_converted_data);  
} else {  
    mBliSrc->process((int32_t *)pRawDataLinearBuf, &kNumRawData,  
                    (int32_t *)pSrcDataLinearBuf, &num_converted_data);  
} //pRawDataLinearBuf经过从采样  
// 后变成pSrcDataLinearBuf  
RingBuf_copyFromLinear(&mSrcDataBuf, pSrcDataLinearBuf, num_converted_data); //pSrcDataLinearBuf给到  
mSrcDataBuf
```

```
RingBuf_copyFromRingBuf(&mProcessedDataBuf, &mSrcDataBuf, freeSpace); //mSrcDataBuf给到ringbuffer  
// (mProcessedDataBuf)
```

AudioALSCaptureDataClientChannelSelector::read(void *buffer, ssize_t bytes) 给到上层in_read接口

```
char *pWrite = (char *)buffer;  
  
RingBuf_copyToLinear((char *)pWrite, &mProcessedDataBuf, ReadDataBytes); //ringbuffer  
// (mProcessedDataBuf) 给到read的buffer
```

PVT

Thank You

www.pvetec.com